Fault-Tolerance Scaffold Simulator

Guided Exercises

Companion to: *Fault-Tolerance Scaffold Simulator: Hardware Errors, Measurement Errors, and Software Decoding* (technical brief).
Purpose: Build intuition for fault tolerance as a hardware–measurement–software co-design problem through structured exploration of the classical repetition-code simulator.

Boris Kiefer
New Mexico State University
bkiefer@nmsu.edu
GitHub
LinkedIn

Introduction

These exercises accompany the Fault-Tolerance Scaffold Simulator and its technical brief. They are organized around the four central lessons of the simulator:

1. Redundancy combined with software decoding can suppress logical failure below any target rate, provided hardware errors are below threshold.
2. The $p = 0.5$ boundary marks total randomness, but practical fault tolerance is governed by much stricter logical-error targets.
3. Sustaining logical fidelity over many correction cycles requires the per-cycle error rate to stay below threshold for every cycle.
4. A resource-estimation curve directly answers how many physical bits are needed to reach a target logical error rate for a given hardware error p.

Each lesson includes pre-exercise questions to surface prior intuitions, guided simulator activities, reflection prompts, and post-exercise questions. Pre- and post-exercise answers are collected in Appendices A and B.

> **How to use these exercises.** Answer all pre-exercise questions *before* opening the simulator. Work through the guided steps, recording what you observe. Then answer the post-exercise questions and compare your answers with the appendix.

Simulator starting settings. Unless a step specifies otherwise, use: $N = 7$, $p = 0.05$, $q = 0.01$, $k = 3$, trials$= 10000$, seed$= 7$, targets 10^{-2}, 10^{-3}, 10^{-4}.

Lesson 1: Redundancy and the Majority-Vote Decoder

Background

A logical bit is encoded into N repeated physical bits. Hardware flips each physical bit with probability p. Measurement corrupts each readout with probability q. A majority-vote decoder recovers the logical value from the noisy evidence. The key question is: does adding more physical bits always help?

Pre-exercise question 0.1. If each physical bit fails independently with probability $p = 0.05$, and you encode a logical bit into $N = 7$ physical bits, do you expect the logical failure rate to be higher or lower than 0.05? Explain your reasoning.

(Answer in Appendix A, question 0.1.)

Pre-exercise question 0.2. Suppose you increase N from 7 to 15 while keeping $p = 0.05$. Do you expect the logical failure rate to decrease, increase, or stay the same? Why?

(Answer in Appendix A, question 0.2.)

Guided steps.

1. Set the starting settings. Run the simulator and record the analytical logical failure rate shown in the summary header.
2. In Panel (a), identify the curve for $N = 7$. Is it above or below the dashed $p_L = p$ diagonal at $p = 0.05$?
3. Change N to 3, 9, 15, and 31 in turn, running the simulator each time. For each value, record the analytical logical failure rate and note whether the curve for that N lies above or below the $N = 7$ curve in Panel (a).
4. Set $p = 0.45$ (near the information boundary). Repeat the sweep over N. Does increasing N still help? Record what you observe.
5. Return p to 0.05. Set $q = 0$ and compare the analytical failure rate to the case $q = 0.01$. What role does measurement error play?

Reflection. Summarize in one or two sentences: under what conditions does more redundancy reliably reduce logical failure, and when does it not?

Post-exercise question 0.1. After completing the steps, does increasing N always reduce logical failure? What determines whether encoding helps or hurts?

(Answer in Appendix B, question 0.1.)

Post-exercise question 0.2. What is the role of measurement error q in the logical failure rate, and how does it interact with hardware error p?

(Answer in Appendix B, question 0.2.)

Quantum analogy. In a quantum error-correcting code, physical qubits play the role of the repeated physical bits here. Increasing the code distance d (analogous to N) suppresses logical errors when the physical error rate is below threshold. Above threshold, a larger code makes things worse — the same behavior you observed in step 4 above. Measurement error here maps onto noisy syndrome extraction in quantum codes: imperfect readout corrupts the evidence available to the decoder every round.

Lesson 2: The Information Boundary and Practical Targets

Background

At $p = 0.5$, each physical bit is equally likely to be correct or flipped. Majority voting has no usable information and is equivalent to guessing. This is the information boundary. However, this boundary is not the engineering target. Useful fault tolerance requires logical failure rates far below the information limit: 10^{-2}, 10^{-3}, or lower depending on the application.

Pre-exercise question 0.3. The simulator marks $p = 0.5$ as the "information boundary." Does this mean that $p = 0.5$ is the threshold below which fault tolerance works? Explain the difference between the information boundary and a practical engineering target.

(Answer in Appendix A, question 0.3.)

Pre-exercise question 0.4. If your application requires a logical failure rate below 10^{-3} per cycle, and your hardware achieves $p = 0.05$, do you expect to need a large or small N? Make a rough guess before using the simulator.

(Answer in Appendix A, question 0.4.)

Guided steps.

1. Use the starting settings ($N = 7$, $p = 0.05$, $q = 0.01$, $k = 3$). In Panel (a), locate the vertical red dotted line at $p = 0.5$ and the dashed $p_L = p$ diagonal. Identify which N curves cross the diagonal at the smallest p.
2. In Panel (b), locate the red contour lines corresponding to targets 10^{-2}, 10^{-3}, and 10^{-4}. At your operating point $p = 0.05$, $q = 0.01$: which targets are met by $N = 7$ and $k = 3$?
3. Increase p to 0.10 and re-run. Do the target contours shift? At $p = 0.10$, $q = 0.01$, which targets are still met by $N = 7$?

4. Set $p = 0.49$ (just below the information boundary). Note the logical failure rate from the summary header. Is it near 0.5? What does majority vote produce here?
5. Set $p = 0.51$ (just above). Note the logical failure rate. What happens to the decoder above the information boundary?

Reflection. In your own words, explain why the $p = 0.5$ boundary is pedagogically important but not the number that matters for engineering a fault-tolerant system.

Post-exercise question 0.3. What is the practical difference between the information boundary at $p = 0.5$ and a target such as $p_L < 10^{-3}$? Which one governs hardware design, and why?

(Answer in Appendix B, question 0.3.)

Post-exercise question 0.4. Panel (b) shows a heatmap of the joint (p, q) operating region. What does the black dashed contour ($p_L = p$) represent, and why is it useful as a reference?

(Answer in Appendix B, question 0.4.)

Quantum analogy. In quantum error correction the analogous boundary is the fault-tolerance threshold. Below threshold, larger codes suppress logical errors exponentially. Above threshold, they amplify errors. However, just as here, the threshold is not the engineering target: hardware roadmaps for surface codes aim for physical error rates of 10^{-3} or lower to reach logical error rates of 10^{-6} or better within a practical code distance. The distinction between the information boundary and the application target is exactly the same systems concept.

Lesson 3: Sustained Correction Over Many Cycles

Background

A single correction cycle is not enough to model real fault-tolerant operation. Useful quantum computation requires error correction to run continuously for many cycles. Fresh hardware errors are introduced at the start of every cycle. The per-cycle logical failure rate p_L accumulates: even a small per-cycle rate becomes significant over many cycles. The multi-cycle failure probability is approximated by

$$P_{\text{fail}}(k) = 1 - (1 - p_L)^k.$$

Pre-exercise question 0.5. If the per-cycle logical failure rate is $p_L = 10^{-3}$, what is your rough estimate of the total logical failure probability after $k = 100$ cycles? After $k = 1000$ cycles? (You may calculate or estimate.)

(Answer in Appendix A, question 0.5.)

Pre-exercise question 0.6. Suppose you want to keep the total logical failure probability below 10^{-2} over $k = 100$ cycles. What per-cycle logical failure rate p_L is required? Does this change your view of how small p_L needs to be?

(Answer in Appendix A, question 0.6.)

Guided steps.

1. Use the starting settings with $k = 1$. Record the analytical logical failure rate.
2. Increase k to 3, 10, 30, and 100 in turn, keeping all other settings fixed. For each k, record:
   - the analytical logical failure rate from the summary header;
   - the position of the $N = 7$ curve in Panel (a) relative to the target lines.
3. At $k = 100$, does $N = 7$ still meet the 10^{-3} target? If not, what N is required? Use Panel (a) or the resource table printed below the figure.
4. Now set $p = 0.10$ and repeat the k sweep. At what k does the 10^{-2} target first fail for $N = 7$?
5. Return to $p = 0.05$, $k = 10$. Gradually increase q from 0.01 to 0.10. At what q does the 10^{-3} target fail for $N = 7$?

Reflection. Fault tolerance is sometimes described as a one-time fix: build a good enough physical device and the logical qubit will be reliable. What do these exercises reveal about that description?

Post-exercise question 0.5. How does increasing the number of correction cycles k affect the curves in Panel (a)? What does this imply about the per-cycle logical failure rate required for a long computation?

(Answer in Appendix B, question 0.5.)

Post-exercise question 0.6. In the multi-cycle model, fresh hardware errors are introduced every cycle. Why does this mean that the corrected physical state at the end of one cycle is not the same as the initial encoded state? What accumulates if correction is not sufficient?

(Answer in Appendix B, question 0.6.)

> **Quantum analogy.** In a fault-tolerant quantum computer, error correction runs continuously throughout the computation. Every gate, measurement, and idle period introduces fresh errors. The logical circuit depth plays the role of the cycle count k here. This is why architecture proposals such as the IonQ Walking Cat framework are organized around the question of how many physical operations are needed to sustain a target logical error rate over a meaningful circuit depth — the same resource question as steps 2–3 above, but for a quantum code.

Lesson 4: Resource Estimation

Background

Given a hardware error rate p, a measurement error rate q, and a cycle budget k, the resource-estimation panel answers a concrete engineering question: what is the minimum code length N needed to keep the total logical failure probability below a target p_{target}? This is the central quantity in hardware roadmap planning. A small improvement in hardware error rate can dramatically reduce the physical-bit overhead needed to hit a logical target.

Pre-exercise question 0.7. If you improve the hardware error rate from $p = 0.10$ to $p = 0.05$ (a factor of two), do you expect the required code length N to decrease by roughly a factor of two as well? Or more, or less? Explain your intuition.

(Answer in Appendix A, question 0.7.)

Pre-exercise question 0.8. Hardware roadmap documents often cite statements such as “we need approximately 1000 physical qubits per logical qubit at current error rates.” What three inputs do you think determine that number?

(Answer in Appendix A, question 0.8.)

> **Guided steps.**
>
> 1. Use the starting settings. In Panel (c), identify the solid curve (your selected $p = 0.05$) and the dashed curves for other p values. For each p value shown, read off the minimum N required to cross the 10^{-3} target line.
> 2. Record these $(p, N_{\min})$ pairs in a small table. Is the relationship between p and $N_{\min}$ linear? Faster or slower than linear?
> 3. Set $k = 1$ and repeat step 1. Then set $k = 10$ and repeat. How does the cycle count affect the required N for the same target?
> 4. In Panel (c), add $p = 0.15$ to the resource p values if it is not already shown. At $p = 0.15$ and $k = 3$, can the 10^{-4} target be reached within the displayed N range? What does this say about operating close to threshold?
> 5. Set $q = 0.05$ (higher measurement error). For $p = 0.05$, how does the required N for the 10^{-3} target change compared to $q = 0.01$? Which error source — hardware or measurement — has more leverage at these parameter values?

Reflection. Panel (c) is sometimes called the “hardware roadmap curve.” Explain in one paragraph how an engineer would use this curve to set a hardware development target, given a fixed application error budget and a fixed circuit depth.

Post-exercise question 0.7. From Panel (c), how does the minimum required code length $N_{\min}$ scale as p approaches threshold from below? Is a small improvement in hardware error rate near threshold worth more or less than the same improvement far below threshold?

(Answer in Appendix B, question 0.7.)

Post-exercise question 0.8. The resource curve depends on three parameters: p, q, and k. If you could improve only one of these by a factor of two, which would give the greatest reduction in required N at the operating point $p = 0.05$, $q = 0.01$, $k = 10$? Use the simulator to check your answer.

(Answer in Appendix B, question 0.8.)

> **Quantum analogy.** Panel (c) is the classical version of the physical-to-logical qubit overhead calculation that drives quantum hardware roadmaps. In a surface code, the code distance d (analogous to N here) determines both the suppression of logical errors and the number of physical qubits required ($\sim 2d^2$ per logical qubit). The crossing-point calculation you performed in steps 1–2 is structurally identical to the calculation behind statements such as "at physical error rate 10^{-3}, distance $d = 7$ is sufficient for a logical error rate of 10^{-6}." The classical scaffold makes the tradeoff visible without requiring knowledge of stabilizer codes or syndrome extraction.

Acknowledgment

This material was developed and/or adapted with support from the National Science Foundation through the QCAP-Pilot and QCAP-Design efforts under NSF Award Nos. OSI-2410813 and OSI-2531569. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Answers to Pre-Exercise Questions

Lesson 1

1.1. With $p = 0.05$ and $N = 7$, the logical failure rate is much lower than 0.05. Majority vote fails only when four or more of the seven bits are flipped, which requires multiple independent errors simultaneously. The probability of four or more independent $p = 0.05$ flips out of seven is very small — roughly 10^{-4} analytically. This is the coding gain: redundancy suppresses logical failure well below the physical error rate when p is small.

1.2. Increasing N from 7 to 15 reduces the logical failure rate further. Majority vote requires more than half the bits to be wrong, and with $p = 0.05$ that is increasingly unlikely as N grows. The improvement is approximately exponential in N when p is well below threshold.

Lesson 2

2.1. The information boundary at $p = 0.5$ is where majority voting loses all information, not where fault tolerance becomes useful. The engineering target is the logical failure rate required by the application, which is typically 10^{-2}, 10^{-3}, or lower. These targets correspond to p values well below 0.5 — often $p < 0.1$ in practice. The information boundary is a theoretical limit; the practical operating regime is far to the left of it.

2.2. At $p = 0.05$, $N = 7$ already meets the 10^{-3} target for $k = 3$ cycles (approximately). A small N suffices when p is comfortably below threshold. If p were near 0.2, a much larger N would be needed.

Lesson 3

3.1. Using $P_{\text{fail}}(k) = 1 - (1 - p_L)^k$ with $p_L = 10^{-3}$: after $k = 100$ cycles, $P_{\text{fail}} \approx 1 - (0.999)^{100} \approx 0.095$, or roughly 9.5%. After $k = 1000$ cycles, $P_{\text{fail}} \approx 1 - (0.999)^{1000} \approx 0.632$, or about 63%. A per-cycle rate of 10^{-3} is not small enough for a 1000-cycle computation.

3.2. From $1 - (1 - p_L)^{100} \leq 10^{-2}$, we get $(1 - p_L)^{100} \geq 0.99$, so $p_L \leq 1 - 0.99^{1/100} \approx 10^{-4}$. The per-cycle logical failure rate must be about 10^{-4} to keep the 100-cycle total below 10^{-2}. This is a factor of ten more demanding than the per-cycle target in isolation.

Lesson 4

4.1. The reduction in required N is more than linear when moving from $p = 0.10$ to $p = 0.05$. Because logical failure is suppressed approximately exponentially in N for p below threshold, the crossing point of a target line with the resource curve moves disproportionately to smaller N as p decreases. Near threshold the required N grows very rapidly; far below threshold it grows slowly.

4.2. The three inputs that determine physical-qubit overhead are: the hardware (physical) error rate p, the logical error target p_{target}, and the cycle budget k (circuit depth). A fourth input, measurement/readout error q, also enters because noisy syndrome extraction reduces decoder performance and effectively increases the required code size.

Answers to Post-Exercise Questions

Lesson 1

1.1. Increasing N does not always reduce logical failure. It helps when the effective observed error rate $r = p+q-2pq$ is below 0.5, so that each bit is more likely correct than wrong. When $r > 0.5$ (above threshold), majority vote is systematically wrong, and adding more bits makes logical failure worse. At $p = 0.45$ with nonzero q, r exceeds 0.5 and larger N increases logical failure.

1.2. Measurement error q increases the effective error rate seen by the decoder from p to $r = p + q - 2pq$. Even if hardware is perfect ($p = 0$), nonzero q introduces logical failures. The two error sources compound: with $p = 0.05$ and $q = 0.01$, $r \approx 0.059$, noticeably larger than p alone. Reducing q is therefore also a hardware design goal, not only reducing p.

Lesson 2

2.1. The information boundary at $p = 0.5$ marks where the decoder has no useful information. The engineering target $p_L < 10^{-3}$ marks where the application can tolerate failure. Hardware design is governed by the application target: the device must operate far enough below threshold that the required code length is achievable and the logical failure rate is below the application budget. The information boundary is a theoretical limit that no practical system approaches.

2.2. The black dashed $p_L = p$ contour marks where encoding and decoding provide no benefit over a single unencoded physical bit. Below the contour (smaller p and q), the code helps. Above it, the overhead of encoding is wasted or counterproductive. It is a useful engineering reference: the operating point should be well below this contour for the code to justify its overhead.

Lesson 3

3.1. Increasing k raises every curve in Panel (a) uniformly, because each additional cycle adds a fresh independent chance of logical failure. For a long computation, p_L must be small enough that $(1 - p_L)^k$ remains close to 1. This requires $p_L \ll 1/k$, so the per-cycle target becomes more stringent as the computation grows longer.

3.2. After majority-vote correction, the physical state is reset to the decoded value. If the decoded value is wrong (a logical failure), the new state encodes the wrong logical bit, and all subsequent cycles start from this corrupted state. Fresh hardware errors in the next cycle compound with any residual damage not corrected in the previous cycle. If p_L is not small, errors accumulate faster than correction removes them.

Lesson 4

4.1. As p approaches threshold from below, the resource curve flattens and the crossing point with any target line moves to very large N. Near threshold, even a small improvement in p moves the crossing point dramatically. Far below threshold, the same absolute improvement in p changes $N_{\min}$ by only a small amount. This means hardware investment is most leveraged when it reduces p from near-threshold to comfortably below threshold.

4.2. At $p = 0.05$, $q = 0.01$, $k = 10$, hardware error p dominates because $p \gg q$. Halving p to 0.025 reduces the effective error rate r more than halving q to 0.005. The simulator confirms this: the resource curve shifts more sharply to smaller N when p is halved than when q is halved at these parameter values. When p and q are comparable, both matter equally.